

Olmran Item Builder

Full Usage Guide - v5.3.1

A tool for parsing Olmran/MUD-style game logs, extracting loot data, building a master equipment database, and finding the optimal gear set for your character.

This guide covers every tab and feature currently in the program. It's a companion to the README on GitHub, written for reading start-to-finish or jumping to the section you need.

Contents

1. Getting Started
2. Parse Tab
3. Fields Tab
4. Export Tab
5. Build Tab
 - 5.1 Master Database File
 - 5.2 Basic Constraints
 - 5.3 Armor Constraints
 - 5.4 Weapon Constraints
 - 5.5 Bank Build
 - 5.6 Shared Search Controls
6. Results Tab
7. Saved Builds Tab
8. Area Items Tab
9. Tips and Example Builds
10. Troubleshooting

1. Getting Started

- Download **OlmranItemBuilder.exe** from the download page and put it anywhere on your computer.
- Double-click it to run - no Python install, no admin rights, no setup step.
- Windows SmartScreen may warn about an unrecognized app the first time (it isn't code-signed yet) - click "More info" then "Run anyway".
- It's a PyInstaller "onefile" build, so it unpacks itself to a temp folder on every launch - expect a few seconds of startup delay each time, not just the first.
- The bundled community equipment list (*Olmran_Community_Eq_and_Stats_List.xlsx*) ships inside the .exe itself and loads automatically on startup.

2. Parse Tab

Load your game logs here and turn them into structured Chat, Combat, and Loot data.

- Load CHAT logs and ACTION logs - the file type is auto-detected from the filename.
- **Search Logs** finds every time an item actually dropped across a whole batch of loaded logs, independent of running a full parse:
- **Drops only** (default, on) - matches real drop events the same way the Loot parser does, so quality/material prefixes ("bright", "glowing", "shining", etc.) don't cause misses or false confidence. For each drop it shows a **snapshot**: every line from the last timestamp seen through the drop line itself, so you can see the fight and kill that led up to it. Click a result row to view its snapshot.
- Uncheck **Drops only** to instead do a plain raw-text search of every loaded file's lines (any mention of the term, not just drops), showing filename, line number, and timestamp per match.
- Case-sensitive matching is optional in both modes.
- Parse options (Chat / Combat / Loot) automatically enable or disable based on which file types are loaded.
- Use the Snapshot buttons to preview parsed data before exporting.
- Export everything to a formatted Excel workbook.

3. Fields Tab

- Customize which fields get extracted for Chat, Combat, and Loot.
- Add, edit, remove, and reorder fields per data type.
- "Reset to Defaults" restores the built-in field set.

4. Export Tab

- Create a new Master Loot Database or append to an existing one (fodder items excluded automatically).

- Export Combat and Loot as separate files if needed.
- Optional summary sheet with run statistics.
- Each chat log gets its own sheet on export.

5. Build Tab

This is where you search a master equipment database for the best gear for your character. It's organized into four inner sub-tabs: Basic Constraints, Armor Constraints, Weapon Constraints, and Bank Build. A shared control area at the bottom (Min/Max/Specific Level, and the search buttons) stays visible no matter which sub-tab is active.

5.1 Master Database File

- **Browse...** - pick your own .xlsx database.
- **Create New** - start a blank one.
- **Use Community List** - instantly loads the bundled community equipment list.
- **Load** - reads the selected file into memory for searching.

Items whose Area is "Class" are excluded from every search entirely at load time (they're role-specific class-quest rewards, not found in the open world) - see the Class Items dropdown below for the one way to still use one deliberately.

5.2 Basic Constraints

Desired Spells - pick spells from category dropdowns instead of typing them, each paired with a tier dropdown ((any), i, ii, iii):

- **Basic** - Agility, Bless, Dexterity, Constitution, Intelligence, Wisdom, Strength, Evade, Combat, Tough.skin
- **Shields/Bufs** - Protect, Blur, Shield, Vitalize, Regenerate, Bleed.resist, Disease.resist, Poison.resist
- **Class Specific** - 23 combat/skill-enhance spells (Reverb, Aura, Backstab, Bash, Berzerk, Crush, Direct, Double, Fired, Improve, Leathers, Lockpick, Mark, Martialarts, Maul, Melee, Pummel, Rake, Repair, Slash, Tap, Thrust, Track, Volley - none of these go above tier ii)
- **General Skills** - Platemail, Leathers, Thrust, Chaos.crush, Slash, All.weapons, Weapons, Climb, Hide, Jump, Swim, Percept, Sneak
- **Protects** - the 8 elemental/mental protects (Cold/Earth/Elemental/Fire/Lightning/Mental/Shock/Water), with their own minor/normal/improved tier labels instead of i/ii/iii

Some spells restrict which tiers are selectable (Agility/Bless cap at ii; Class Specific spells cap at ii; Disease.resist alone goes up to tier iv) - the tier dropdown greys out anything that spell can't reach. Click **Add to List** to add the current dropdown selection as a removable chip under **Wanted Spells**. Requesting the same spell at two different tiers (e.g. Dexterity i and Dexterity iii) is treated as one requirement, not two.

Wanted Sigils - pick one or more Sigil types (Cold/Earth/Fire/Lightning/Pain/Shock/Water) and the search actively favors armor carrying them, the same way it does for Wanted Spells - useful since many armor

pieces carry a Sigil but no Spell at all. Unlike spells, sigils don't need to be exclusive to one slot - multiple slots can each independently carry a wanted Sigil, and a Wanted Spell always outranks a Wanted Sigil for the same slot.

Priority Spells and Priority Tier - Priority Spells are always actively searched for (not just preferred among items that already matched something else). Priority Tier pairs one specific spell with one specific tier, so the search targets exactly that tier for it, even over a higher tier that's available elsewhere.

Required Items - type a specific item's name under **Require Item:** and click **Add to Build**. It's looked up by exact match first, falling back to a spelling-tolerant fuzzy search that asks you to confirm the closest match if there's no exact hit. Once added it's forced into its slot on every future search, with the rest of the build calculated around it.

Class Items dropdown - lists every item excluded for being in the "Class" area, formatted as *Item-MOB-Spell* and sorted by Mob (Mob shown in bold caps). Selecting one and clicking **Add to Build** forces it into Required Items - the one deliberate way an otherwise-excluded item can still end up in a build.

Level Filtering - Min Level / Max Level restrict to a range; Specific Level restricts to an exact level (mutually exclusive with Min/Max). Leave all three blank for no restriction.

If no match at Specific Level (only enabled once Specific Level has a value) controls what Find Optimal Build does when nothing matches a wanted spell at exactly that level/tier:

- **Go down a tier** - keep the exact level, accept a lower tier if needed
- **Go down in level** - keep the exact tier, search downward for the closest available level
- **Both** - relax level and tier together, preferring the closest combination
- **Don't populate slot** (default) - no fallback; the slot is left empty

Only Found In (realm filter) - check any combination of Evil, Chaos, Good, Glory Bea, Crafted, Event, or Kaid to restrict results to those realms. Leave everything unchecked (or check **All**) to search every realm except Crafted. Kaid has its own column: **Kaid All** matches any Kaid sub-realm, or check specific colors (White/Green/Red/Purple) instead - mutually exclusive with Kaid All. **Crafted items never appear unless the Crafted checkbox is checked**, and a build can include at most one Crafted-realm item even then.

5.3 Armor Constraints

- The **All:** row sets an armor type for every slot at once.
- Override individual slots (Head, Cloak, Body, Hands, Legs, Feet) with Cloth / Leather / Studded / Plate.
- Per-slot **Defense** (range) and **Sigil** dropdowns - soft preferences: a matching item is favored, but a slot is never left empty just because nothing matches.
- Up to 3 slots can be marked **Max Lvl** priority - greedily locks that slot to the highest-level item that still carries a wanted/priority spell, before the normal search runs for everything else.
- **Set as Default** / **Clear Default** / **Clear All** manage your saved preferences, persisted between sessions.

5.4 Weapon Constraints

- **Weapon Types/Combo's** - check whichever your build uses: Dual-Wield 1h, 1h/Shield, 2h/Shield, Fired 1h/Shield, Two-Handed, 1 Claw, 2 Claw. Most have their own Style (Melee/Direct/Parry Staff/Fired) and Damage Type (Slashing/Thrusting/Crushing) dropdowns.
- Leave everything unchecked and Weapon/Shield aren't populated at all. 1 Claw/2 Claw replace Weapon and Shield entirely rather than filling alongside them.
- **Melee Weapon Constraints** - soft-preference Damage/Timer/Fumble/Accuracy/Sigil dropdowns (each with an optional Priority checkbox, capped at 3), plus a **Weight** range - soft preference by default, or check **Hard Filter** to exclude out-of-range weapons outright. Applies to every weapon style and to Claw slots too. 1 Claw/2 Claw each get their own Sigil dropdown (first for claw 1, second for claw 2).
- **Shield Constraints** - Defense and Sigil dropdowns working like Armor Constraints' per-slot versions, plus Cloth/Leather/Studded/Plate checkboxes (a hard filter, same as Armor Constraints' own).

5.5 Bank Build

Paste a bank/inventory listing from the game and use it as the basis for a search, instead of (or alongside) the full database. Two inner tabs share the idea:

- **Import** - paste a listing, type a Character Name, then click Save to Saved Items. The first time a given name is used, a new tab for that character is created automatically under Saved Items; using the same name again just updates that character's existing list. This is the only way items get into Saved Items now - Import doesn't run a search itself.
- **Saved Items** - a notebook of its own. **Main** is a read-only combined view of every character's items, plus anything saved before character tracking existed - just the list, and a Clear Saved List button scoped to that older, character-less leftover data (it never touches any character's own items). **Each character gets its own tab** with its own Only Found In checkboxes, Prioritize Saved Items/Hard Search checkboxes, a **Search all characters** checkbox, a Find Best Bank Build button, and its own Clear Saved List button.

Hard Search holds each wanted spell's exact tier and restricts candidates to that character's items only - a slot with nothing at the exact tier reads "No available item". **Search all characters**, when checked, pools every character's non-Kaid items together for the search - but Kaid items still only come from that one character's own tab, since Kaid gear doesn't drop when you die and represents what that specific character is actually carrying, not the whole roster's stash.

Four paste formats are recognized and can even be mixed in one paste: a numbered Strongbox listing (*12. item name [Level|Slot|...tags]*), the same listing without numbering, an Inventory listing ("*(w)\"(h)*") marked lines always count; an unmarked or stack-count line also counts if it matches a real item in the loaded database), and an Items in use listing (*On Head: item name, Held Left: nothing*).

Each character's list reconciles by **content type** - bank listing vs. inventory/equipped listing - rather than which content was pasted most recently. Updating just a bank paste never wipes out that character's inventory-sourced items, and vice versa; a paste containing both kinds updates both. A second (or further) copy of the same item is listed at the bottom of a list, prefixed "::*extra::*". Everything persists across closing and reopening the program.

The Bank icon appears in the leftmost column of every search's results (Find Optimal Build, Show All Matches, everywhere) whenever an item matches something saved anywhere - any character, or the older

Main-only leftover data.

5.6 Shared Search Controls

- ■ **Find Optimal Build** - exact search across every slot at once for the combination covering as many wanted spells as possible under your constraints.
- ■ **Show All Matches** - lists every item matching your filters, without narrowing to one per slot.
- ■ **Generate multiple build options** - also generates up to 10 alternate full builds by swapping in equally-good tied items slot by slot.

6. Results Tab

- **Display: Best Per Slot / All Matches** toggles between the two search modes above.
- **Build Variant** dropdown - active when multiple build options were generated.
- Columns: Bank icon, Slot, Item, Type, Spell, Sigil, Level, Mob, Area, and **Alt Options** (other items that tied for that slot).
- **Right-click a filled item** to remove it from the build - it stays excluded from every future search until a genuinely new search runs. Two buttons appear once something's been removed:
- **Rebuild (Full Database)** - re-searches everywhere, excluding what's been removed.
- **Rebuild (Saved Items First)** - builds as much of the set as possible from Saved Items, then automatically searches the full database for whatever it can't cover - a complete gear set every time. Useful for PvP: remove a good item you don't want to risk losing on death, rebuild, and get a full set that reuses your bank wherever possible and clearly shows (via the missing Bank icon) exactly what you'd still need to go acquire for the rest.
- **Right-click an empty slot** ("No suitable item found", etc.) offers the same two Rebuild actions.
- **Search Missing Slots (Full Database)** button appears whenever a Saved Items Hard Search leaves gaps - fills them in place from the full database, asking about a lower tier if the exact one isn't found anywhere.
- **Remove Area** - strip all items from a given area out of the current results.
- **Export Results / Save Build** - save the current table to Excel, HTML, image, or text, or pin it as a permanent panel under Saved Builds.

7. Saved Builds Tab

- Click **Save Build** from the Results tab to add the current results as a panel here.
- Each panel has its own renamable name, its own Export As... (Excel/HTML/Image/Text), and a Remove button.
- Saved Builds persist across closing and reopening the program.

8. Area Items Tab

- Pick an Area and browse every item droppable there, straight from the loaded master database.
- The Area field is typeable - type a few letters to narrow the list, or click/tab into it to see the full alphabetical list right away.
- Results are sorted by slot: armor first (Plate, Studded, Leather, Cloth), then Jewel, Shield, Weapon.

9. Tips and Example Builds

Quick start: Use Community List, add the spells you want, set level/armor/weapon/realm constraints as needed, click Find Optimal Build.

- **Tank:** Melee + Plate armor + Weapon + Shield + Slashing + Min Level 60
- **Two-Handed Warrior:** Melee + Plate armor + Two-Handed + Crushing + Level 50-70
- **Dual-Wield Rogue:** Melee + Leather armor + Weapon + Dual-Wield + Thrusting
- **Dual-Claw Fighter:** Melee + 2 Claw + Slashing
- **Caster with Staff:** Direct (Caster) + Cloth armor + Two-Handed + Crushing

10. Troubleshooting

- **"Windows protected your PC" / SmartScreen warning** - expected, the exe isn't code-signed yet. Click "More info" then "Run anyway".
- **The exe takes a few seconds to open every time** - expected; it's a PyInstaller onefile build that unpacks itself to a temp folder on every launch.
- **Files not loading** - check the file extension (.log, .txt, .xlsx), make sure it isn't open in another program, and try copying it somewhere else if the path has unusual characters.
- **Parse results empty** - verify the log actually contains delve output ("You examine X closely") and that CHAT vs ACTION auto-detection picked the right type.
- **Community List not found** - if running from source rather than the .exe, make sure Olmran_Community_Eq_and_Stats_List.xlsx is in the same folder, or use Browse... to point at it manually.
- **Can't see everything if the window is resized smaller** - the whole window scrolls (mouse wheel, or the scrollbar on the right edge) if it's ever shrunk below what its content needs.

Full version history and the latest release are on GitHub: github.com/Gnawbie/Olmran-parser-ItemBuilder